Jai
2301420045
Assignment 1

Abstraction in computer systems means hiding complex details and showing only the necessary features. It helps manage complexity, improves design clarity, and allows easier development and debugging.

Levels of abstraction go from hardware to software: hardware → machine language → assembly → high-level language → applications. Example: when writing Python code, we don't deal directly with circuits or binary.

The basic functional units of a computer are Input Unit, Output Unit, Memory Unit, Arithmetic Logic Unit (ALU), and Control Unit (CU).

ALU performs arithmetic and logical operations, while CU controls and coordinates all operations of the computer.

The Von-Neumann model stores instructions and data in the same memory and uses a single path to access them. The stored-program concept means programs are stored in memory and executed sequentially.

The Von-Neumann bottleneck is the limitation caused by a single data path between CPU and memory, slowing performance.

Moore's Law states that the number of transistors doubles approximately every two years, increasing performance and reducing cost.

IPC (Instructions Per Cycle) is the number of instructions a CPU executes in one clock cycle.

Execution time is the total time to run a program. Clock rate is the number of cycles per second. CPI is cycles per instruction. IPC is the inverse of CPI. Execution time = (Instruction Count × CPI) / Clock Rate.

Data representation uses bits and bytes. Signed numbers represent positive and negative values, while unsigned represent only non-negative values. Example: 8-bit signed range is -128 to 127.

Basic operations on data include arithmetic operations, logical operations, comparison, and data movement.

Characters are represented using ASCII or Unicode. Decimals are represented using floating-point (IEEE 754).

Block Diagram of Computer:

```
[ Input ] ---> [ CPU (ALU + CU) ] --->
[ Output ]
|
[ Memory ]
```

Von-Neumann Architecture:

```
[ Input ] [ Output ]
\ /
```

[CPU]
|
[Memory]
(Single memory for data + instructions)

Performance Equation Map:

Instruction Count (IC)
|
v
CPI ---------> IPC = 1/CPI
|
v
Clock Rate ---------> CPU Time = (IC × CPI) / Clock Rate

4-bit 2's Complement Range:

-8 -7 -6 -5 -4 -3 -2 -1 0 1 2 3 4 5 6 7

CPU A: (3.0 GHz / 1.5 = 2.0), CPU B: (2.2 GHz / 1.0 = 2.2). CPU B is faster due to higher instruction throughput.

Cache misses increase stalls, reducing IPC and increasing execution time. Fix: improve cache size or locality.

Use fixed-point representation by scaling (multiply by 10) to store values from -50 to +150 with 0.1 precision efficiently.

Moore's Law no longer guarantees automatic performance gains due to physical limits, power constraints, and need for architectural improvements.